Data Structures, Files & JSON

Session 09 practical workbook · Python 3 · VS Code · Sales order records

In Session 08 the program processed one order at a time and forgot it as soon as it finished. Real data arrives as many records: ten orders today, a thousand tomorrow. This session gives Python the shapes to hold them (**lists, dictionaries, tuples, sets**), a concise way to transform them (**comprehensions**), and ways to keep and share them (**files, CSV and JSON**).

Every example was run on Python 3 and the output shown is the real output. The session ends with a mini-pipeline that loads ten orders from a CSV file, enriches them, saves them as JSON, and checks the result.

The four data structures at a glance

Structure	Example	Purpose	Mutable?	When to use it
List	`[120.5, 89.99, 450.0]`	An ordered collection of items. Duplicates are allowed.	Yes	Many items where order matters: today's transactions, a set of orders, a column of values
Dictionary	`{"name": "Thandi", "balance": 15250.0}`	Named fields: each **key** points to a **value**. Keys are unique.	Yes	One record whose fields need names: a customer, an account, an order
Tuple	`("ORD-1001", "West")`	A small, fixed group of values in set positions.	No (positions cannot be reassigned)	A fixed pair or group that should not change, e.g. coordinates or an ID-region pair; unpacking
Set	`{"West", "North", "East"}`	Unique values only, with fast "is it in here?" checks. No positions.	Yes	Removing duplicates; checking membership, e.g. distinct regions or products
List of dictionaries	`[{"id": "A1", ...}, {"id": "A2", ...}]`	Many records, each with named fields. The shape of a table.	Yes	A dataset: each dictionary is a row, each key is a column (the idea behind Pandas)

> **Rule of thumb:** Choose the structure by the behaviour you need: **order** (list, tuple), **named fields** (dictionary), **fixed positions** (tuple), **uniqueness** (set).

Part	Topic	Business case used to explain it
1	Lists	A cashier's transactions; a product catalogue
2	Dictionaries	One bank customer record; one sales order
3	Lists of dictionaries	A small batch of orders
4	Tuples, sets and choosing a structure	Fixed ID-region pairs; distinct sales regions
5	Comprehensions	Filtering large orders; collecting customer names
6	Files and paths	Saving a note that survives after the program ends
7	CSV	Loading ten orders from a spreadsheet export
8	JSON	Sharing the enriched orders with another system
9	Integrated practical	Sales Order Records mini-pipeline

Lesson plan (3 hours)

Introduce each structure with the slides, then move to VS Code. Keep asking one question all session: **what shape should this data have?**

Time	Block	You do (trainer)	Learners do	Slides
0–10	Starting point	Show one Session 08 order, then ask: what if there are ten? Walk the four needs and the six-step line.	Name a need for each structure.	1–4
10–30	Part 1 · Lists	Predict `[0]`, `[-1]`, `[1:3]`. Update, append, loop.	Try it yourself 1.	5–11
30–55	Parts 2–3 · Dictionaries and records	Position 3 versus `"unit_price"`. KeyError versus `.get()`. Trace `orders[0]["customer_name"]`.	Try it yourself 2 and 3.	12–20
55–80	Part 4 · Tuples, sets, choice	Unpacking; unique regions; the structure choice table.	Try it yourself 4.	21–29
80–90	Break			30
90–110	Part 5 · Comprehensions	Always show the loop first, then the comprehension.	Try it yourself 5.	31–37
110–125	Part 6 · Files and paths	Memory versus disk; `with open`; FileNotFoundError.	Try it yourself 6.	38–44
125–145	Part 7 · CSV	Values arrive as text. DictReader, then convert.	Try it yourself 7.	45–54
145–175	Parts 8–9 · JSON and practical	`json.dump`, the set failure, the round trip. Then coach the pipeline.	Complete the starter file; verify.	55–69
175–180	Close	The lifecycle and the ten questions.	Reflection questions.	70–73

Before class

- ☐ A new folder `s09_data_structures` opened in VS Code with **File > Open Folder**.
- ☐ `S09_orders.csv` and `s09_pipeline_starter.py` from the class bundle copied into that folder.
- ☐ Learners know that VS Code runs scripts from the **opened folder**. If they open a parent folder instead, `S09_orders.csv` will not be found.

Teaching moves for this session

Move	How it sounds in class
Name the shape	"Say it in words: this is a list of dictionaries, mostly text."
Trace from the outside in	"What does `orders[0]` give you? Now what does `["customer_name"]` give you?"
Check the type at the boundary	"It came from a CSV. What type is `unit_price` right now?"
Loop first	"Write the loop. Only then turn it into a comprehension, if it stays readable."
Verify the file	"The file exists. How do you know it is right?"

Starting point: from one order to many

start_one_order.py

```
order_id = "ORD-1001"
customer_name = "Northstar Retail"
unit_price = 250.0
quantity = 4
discount_pct = 10.0
tax_pct = 5.0
print(order_id, customer_name, unit_price * quantity)
```

OUTPUT

```
ORD-1001 Northstar Retail 1000.0
```

One order needs six variables. Ten orders would need sixty, and a loop could not say "every order". Four needs appear:

Need	Why	Answer
Hold many values	many orders must live in one object a loop can process	a **list**
Name the fields	price and quantity should be read by name, not by remembered position	a **dictionary**
Keep the data	Python objects disappear when the program ends	a **file** (text, CSV)
Share the result	other programs need data that keeps its structure	**JSON**

Single value → **Collection** → **Named record** → **Collection of records** → **Persistent file** → **Exchange format**

This line is the map for the whole session: each part moves one step to the right.

Part 1 Lists

A **list** holds many values in one object, in order. It is written with square brackets and commas.

> **Business case:** A cashier's till records every sale. Rather than one variable per sale, keep them in one list.

p1_till.py

```
till_sales = [120.50, 89.99, 450.00, 35.25, 89.99]

print(till_sales)
print("Number of sales:", len(till_sales))
print("Total:", round(sum(till_sales), 2))
print("Largest sale:", max(till_sales))
```

OUTPUT

```
[120.5, 89.99, 450.0, 35.25, 89.99]
Number of sales: 5
Total: 785.73
Largest sale: 450.0
```

Three things to notice: the list keeps the **order** the sales happened in, **duplicates are allowed** (89.99 appears twice and both count), and built-in functions such as `len()`, `sum()` and `max()` work on the whole list at once.

Indexing and slicing

PREDICT FIRST

Before running, write down what each of the three lines will print.

p1_index.py

```
product_names = [
    "Analytics Starter Kit",
    "Forecasting Toolkit",
    "Analytics Starter Kit",
    "Workflow Essentials",
]

print(product_names[0])
print(product_names[-1])
print(product_names[1:3])
```

OUTPUT

```
Analytics Starter Kit
Workflow Essentials
['Forecasting Toolkit', 'Analytics Starter Kit']
```

Expression	Meaning	Result
`product_names[0]`	position 0 is the **first** item	Analytics Starter Kit
`product_names[-1]`	negative positions count from the end; -1 is the last	Workflow Essentials
`product_names[1:3]`	a **slice**: start at 1, stop **before** 3	a new list of 2 items

Indexing works exactly like the string indexing from Session 07, but each position now holds a whole value instead of one character.

Lists are mutable: update and append

p1_mutable.py

```
products_for_practice = ["Starter Pack", "Forecasting Toolkit"]
products_for_practice[0] = "Analytics Starter Kit"
products_for_practice.append("Customer Insights Bundle")
print(products_for_practice)
```

OUTPUT

```
['Analytics Starter Kit', 'Forecasting Toolkit', 'Customer Insights Bundle']
```

Mutable means the list can change after it is created. Assigning to a position replaces that item; `.append()` adds one item at the end.

Loop over the items directly

p1_loop.py

```
products_for_practice = ["Analytics Starter Kit", "Forecasting Toolkit",
"Customer Insights Bundle"]

for product_name in products_for_practice:
    print(product_name)
```

OUTPUT

```
Analytics Starter Kit
Forecasting Toolkit
Customer Insights Bundle
```

In Session 08 the loop variable came from `range()`. Here it is each **item** of the list in turn, so no position numbers are needed.

> **In data science:** A list is the natural home for a column of values: daily sales, customer ages, sensor readings. NumPy arrays and Pandas columns (Sessions 11–12) build on exactly this idea.

TRY IT YOURSELF

Start with `monthly_sales = [42000, 38500, 51000, 47250]` (January to April).

(a) Print January and April using indexing. (b) Print February and March with one slice. (c) February is corrected to 39000: update it. Then append May's sales of 49800. (d) Loop over the list and print each value, then print the total with `sum()`.

Part 2 Dictionaries

A list remembers values by **position**. That works for a list of similar things, but not for the different fields of one record.

p2_problem.py

```
order_values = ["DEMO-01", "Northstar Retail", "West", 250.0, 4]
print(order_values[3])
```

OUTPUT

```
250.0
```

`250.0`, but what is position 3? You have to remember. A **dictionary** names every field:

p2_dict.py

```
order_record = {
    "order_id": "DEMO-01",
    "customer_name": "Northstar Retail",
    "region": "West",
    "unit_price": 250.0,
    "quantity": 4,
}
print(order_record["unit_price"])
```

OUTPUT

```
250.0
```

A dictionary is written with curly braces. Each entry is a **key** (the field name, before the colon) and a **value** (the data, after it). List indexes ask *which position?*; dictionary keys ask *which named field?*

> **Watch out:** Keys are unique inside one dictionary. Assigning to an existing key **replaces** its value; it never creates a second field with the same name.

Read, update and add fields

> **Business case:** A bank keeps one record per customer. A deposit updates the balance; a new field records the e-mail address.

p2_account.py

```
account = {
    "account_no": "ACC-2041",
    "name": "Thandi Nkosi",
    "segment": "Premier",
    "balance": 15250.00,
}

print(account["name"])

account["balance"] = account["balance"] + 2000.00
account["email"] = "thandi@example.com"
print(account)
```

OUTPUT

```
Thandi Nkosi
{'account_no': 'ACC-2041', 'name': 'Thandi Nkosi', 'segment': 'Premier', 'balance':
17250.0, 'email': 'thandi@example.com'}
```

The same three moves work on the order record from the deck, including adding a **derived field** calculated from other fields:

p2_derived.py

```
order_record = {"order_id": "DEMO-01", "customer_name": "Northstar Retail",
                "region": "West", "unit_price": 250.0, "quantity": 4}

order_record["quantity"] = 5
order_record["estimated_subtotal"] = order_record["unit_price"] * order_record["quantity"]
print(order_record)
```

OUTPUT

```
{'order_id': 'DEMO-01', 'customer_name': 'Northstar Retail', 'region': 'West',
'unit_price': 250.0, 'quantity': 5, 'estimated_subtotal': 1250.0}
```

Missing keys: required or optional?

p2_keyerror.py

```
order_record = {"order_id": "DEMO-01", "unit_price": 250.0, "quantity": 4}
print(order_record["sales_rep"])
```

ERROR

```
Traceback (most recent call last):
  File "C:\Users\student\s09_data_structures\p2_keyerror.py", line 2, in <module>
    print(order_record["sales_rep"])
          ~~~~~~~~~~~~^^^^^^^^^^^^^
KeyError: 'sales_rep'
```

A missing **required** field raises a `KeyError`, and that is useful: it tells you the record is incomplete. For an **optional** field, `.get()` supplies a default instead:

p2_get.py

```
order_record = {"order_id": "DEMO-01", "unit_price": 250.0, "quantity": 4}
sales_rep = order_record.get("sales_rep", "Not assigned")
print("Sales representative:", sales_rep)
```

OUTPUT

```
Sales representative: Not assigned
```

> **Watch out:** Use `.get()` only when a missing value is genuinely acceptable. A default for a required field such as `unit_price` would hide a real data problem.

Looking at every field

p2_items.py

```
account = {"account_no": "ACC-2041", "name": "Thandi Nkosi", "balance": 15250.0}

for key, value in account.items():
    print(key, "->", value)
```

OUTPUT

```
account_no -> ACC-2041
name -> Thandi Nkosi
balance -> 15250.0
```

`.items()` gives each key and value together. It is handy for inspecting a record; normal processing code is clearer when it names the fields it needs.

TRY IT YOURSELF

Create a dictionary `employee` with `employee_id` "E-118", `name` "Sipho Dlamini", `department` "Finance" and `salary` 32000.0.

(a) Print the department. (b) Give a 5% increase by updating `salary`. (c) Add a new field `manager` with the value "L. Mokoena". (d) Use `.get()` to print `employee`'s "phone", showing "Not on file" if it is missing.

Part 3 Lists of dictionaries

Put the two together and you get the most important shape in this session: a **list** of records, where each record is a **dictionary**.

p3_records.py

```
mini_orders = [
    {"order_id": "DEMO-01", "customer_name": "Northstar Retail", "region": "West",
"product_name": "Analytics Starter Kit"},
    {"order_id": "DEMO-02", "customer_name": "Bluewave Studio", "region": "North",
"product_name": "Forecasting Toolkit"},
    {"order_id": "DEMO-03", "customer_name": "Greenfield Foods", "region": "West",
"product_name": "Analytics Starter Kit"},
]

print(len(mini_orders))
print(mini_orders[0])
print(mini_orders[0]["customer_name"])
```

OUTPUT

```
3
{'order_id': 'DEMO-01', 'customer_name': 'Northstar Retail', 'region': 'West',
'product_name': 'Analytics Starter Kit'}
Northstar Retail
```

Trace from the outside in

Expression	Step	Result
`mini_orders[0]`	list indexing returns one item, and that item is a dictionary	the first order record
`mini_orders[0]["customer_name"]`	then the key reads one field of that record	`Northstar Retail`

Do not read a nested expression all at once. Take one layer at a time.

Loop through the records

p3_loop.py

```
for order in mini_orders:
    print(order["order_id"], "-", order["customer_name"])
```

OUTPUT

```
DEMO-01 - Northstar Retail
DEMO-02 - Bluewave Studio
DEMO-03 - Greenfield Foods
```

This snippet uses `mini_orders` from the previous example.

On every pass, `order` is one whole dictionary, so each line of the loop can use fields by name. Combine it with an `if` from Session 08 to pick out records:

p3_filter.py

```
for order in mini_orders:
    if order["region"] == "West":
        print(order["order_id"], order["product_name"])
```

OUTPUT

```
DEMO-01 Analytics Starter Kit
DEMO-03 Analytics Starter Kit
```

In data science: A list of dictionaries is a table: each dictionary is a **row**, each key is a **column**. When you meet Pandas DataFrames in Session 12, this is the mental model to keep.

TRY IT YOURSELF

Build a list `accounts` with three dictionaries, each with `name` and `balance`: Thandi 15250.0, Pieter 8400.0, Ayesha 22100.0.

(a) Loop and print every name with its balance. (b) Print only the customers whose balance is above 10 000. (c) Use a running total to print the combined balance.

Part 4 Tuples, sets and choosing a structure

Tuples: a small, fixed group

Business case: An order is shipped to one region. The pair (order ID, region) belongs together and should not be edited by accident.

p4_tuple.py

```
order_key = ("DEMO-01", "West")
order_id, region = order_key

print("First tuple item:", order_key[0])
print("Unpacked order ID:", order_id)
print("Unpacked region:", region)
```

OUTPUT

```
First tuple item: DEMO-01
Unpacked order ID: DEMO-01
Unpacked region: West
```

A **tuple** uses round brackets. Like a list it is ordered and can be indexed. **Unpacking** (`order_id, region = order_key`) gives each position its own name in one line.

p4_tuple_error.py

```
order_key = ("DEMO-01", "West")
order_key[0] = "DEMO-99"
```

ERROR

```
Traceback (most recent call last):
  File "C:\Users\student\s09_data_structures\p4_tuple_error.py", line 2, in <module>
    order_key[0] = "DEMO-99"
    ~~~~~~~~~^^^
TypeError: 'tuple' object does not support item assignment
```

Positions in a tuple cannot be reassigned, which is what *immutable* means here. One precise point: if a tuple holds a list, that inner list can still change, because the tuple only fixes **which objects** sit in its positions.

Sets: unique values

> **Business case:** Management asks: which regions did we sell to? The list of orders mentions some regions many times; they want each region once.

p4_set.py

```
regions_with_duplicates = ["West", "North", "West", "East", "North"]
unique_regions = set(regions_with_duplicates)

print("Unique regions:", sorted(unique_regions))
print("How many:", len(unique_regions))
print("Is West present?", "West" in unique_regions)
print("Is South present?", "South" in unique_regions)
```

OUTPUT

```
Unique regions: ['East', 'North', 'West']
How many: 3
Is West present? True
Is South present? False
```

`set()` keeps one copy of each value. The question a set answers best is **membership**: is this value in here? `sorted()` turns it into a list in a stable order for display.

p4_set_error.py

```
unique_regions = {"West", "North", "East"}
print(unique_regions[0])
```

ERROR

```
Traceback (most recent call last):
  File "C:\Users\student\s09_data_structures\p4_set_error.py", line 2, in <module>
    print(unique_regions[0])
          ~~~~~~~~~~~~~~^^^
TypeError: 'set' object is not subscriptable
```

A set has no positions, so it cannot be indexed, and the order it prints in is not guaranteed (your output may show the regions in a different order from a classmate's). Never rely on it for a report; use `sorted()`.

Choosing the right structure

Structure	Main idea	Order / position	Duplicates	Mutable?	Typical use here
List	ordered sequence	yes, by position	allowed	yes	many order records in source order
Dictionary	named key-value mapping	access by key	keys unique	yes	one business record
Tuple	small fixed group	yes, by position	allowed	no reassigning	a fixed pair to unpack
Set	unique membership	no positions	removed	yes	distinct regions, membership checks

Ask yourself	If yes, consider
Does order matter, and can values repeat?	a list
Do the fields need names?	a dictionary
Should the positions stay fixed?	a tuple
Is the question about uniqueness or membership?	a set
Will it be saved as JSON?	remember a set must become a list first (Part 8)

TRY IT YOURSELF

Choose list, dictionary, tuple or set for each need, and give one reason:

(1) Every card swipe at a store today, in the order they happened. (2) One employee's details, read by field name. (3) A branch's fixed (latitude, longitude) location. (4) The distinct ATM IDs that reported an error today, where some ATMs reported several times.

Part 5 Comprehensions

A **comprehension** is a short way to write a loop that builds a new collection. Always understand the loop version first.

Transform: build a list from every item

p5_transform.py

```
upper_product_names = []
for order in mini_orders:
    upper_product_names.append(order["product_name"].upper())
print(upper_product_names)

upper_product_names = [order["product_name"].upper() for order in mini_orders]
print(upper_product_names)
```

OUTPUT

```
['ANALYTICS STARTER KIT', 'FORECASTING TOOLKIT', 'ANALYTICS STARTER KIT']
['ANALYTICS STARTER KIT', 'FORECASTING TOOLKIT', 'ANALYTICS STARTER KIT']
```

Read the comprehension as: *build a list of this expression, for each item in this collection*. Both versions give the same result.

Filter: keep the items that pass a test

p5_filter.py

```
west_orders = [order for order in mini_orders if order["region"] == "West"]
print(len(west_orders))
print([order["order_id"] for order in west_orders])
```

OUTPUT

```
2
['DEMO-01', 'DEMO-03']
```

Filter and transform together

> **Business case:** The sales manager wants the IDs of every large order (final total of 950.0 or more) from the ten enriched orders.

p5_large_ids.py

```
large_order_ids = [
    order["order_id"]
    for order in orders
    if order["final_total"] >= 950.0
]
print(large_order_ids)
```

OUTPUT

```
['ORD-1002', 'ORD-1004', 'ORD-1006', 'ORD-1009']
```

`orders` here is the enriched list built in Part 7; it is loaded behind the scenes for this example.

The bracket decides what gets built

Form	Builds	Level
`[expr for x in items]`	a list	core
`{expr for x in items}`	a set	guided
`{key: value for x in items}`	a dictionary	recognise it

p5_set_dict.py

```
unique_regions = {order["region"] for order in orders}
print(sorted(unique_regions))

region_by_order_id = {order["order_id"]: order["region"] for order in orders}
print(region_by_order_id["ORD-1004"])
```

OUTPUT

```
['East', 'North', 'South', 'West']
West
```

The readability rule

Use a comprehension for a short pattern that reads at a glance. When there are several conditions or steps that deserve names, a loop is clearer:

p5_loop_clearer.py

```
ids = []
for order in orders:
    is_west = order["region"] == "West"
    is_large = order["final_total"] >= 950.0
    if is_west and is_large:
        ids.append(order["order_id"])
print(ids)
```

OUTPUT

```
['ORD-1004', 'ORD-1009']
```

TRY IT YOURSELF

Using the `accounts` list from Part 3:

(a) Build a list of all names in uppercase. (b) Build a list of the names with a balance above 10 000. (c) Add a `segment` field to each account ("Premier", "Standard", "Premier") and build a **set** of the segments. Write each one as a loop first.

Part 6 Files and paths

Everything so far lived in memory and vanished when the program ended. A **file** keeps data after the program stops.

In memory	write → / ← read	On disk
Typed Python objects (lists, dicts, numbers) while the program runs. Gone when it ends.	Writing turns objects into text. Reading turns text back into new objects.	Text in a file, e.g. `S09_orders.csv`. It stays after the program stops.

A file boundary is also a **type** boundary: in a CSV, `250.0` is text until you convert it; in JSON, a Python set has no equivalent.

Paths: where is the file?

```
INPUT_FILE = "S09_orders.csv"
ENRICHED_JSON_FILE = "S09_enriched_orders.json"
SUMMARY_FILE = "S09_summary.txt"
PRACTICE_TEXT_FILE = "S09_file_practice.txt"
LARGE_ORDER_THRESHOLD = 950.0
```

These are **relative paths**: Python looks for them in the current working folder. In VS Code that is the folder you opened. An **absolute path** gives the full location (`C:\Users\...`), but breaks on another computer.

with open(): open, use, close

Part	Meaning
`open(name, mode=..., encoding="utf-8")`	opens the file; utf-8 makes text handling predictable
mode `"r"`	read an existing file
mode `"w"`	write; **replaces** a file with the same name
mode `"a"`	append to the end
`with ... as file:`	closes the file automatically when the indented block ends, even after an error

p6_text_file.py

```
PRACTICE_TEXT_FILE = "S09_file_practice.txt"
practice_message = "A file persists after this program finishes."

with open(PRACTICE_TEXT_FILE, mode="w", encoding="utf-8") as text_file:
    text_file.write(practice_message)

with open(PRACTICE_TEXT_FILE, mode="r", encoding="utf-8") as text_file:
    reloaded_message = text_file.read()

print(reloaded_message)
```

OUTPUT

```
A file persists after this program finishes.
```

A string goes to a file and comes back as a **new** string. After running, `S09_file_practice.txt` appears in the VS Code Explorer; open it to see the text.

When the file is not there

p6_missing.py

```
try:
    with open("missing_S09_orders.csv", mode="r", encoding="utf-8") as missing_file:
        missing_file.read()
except FileNotFoundError as error:
    print("File not found. Check the filename and the current folder.")
    print("Requested path:", error.filename)
```

OUTPUT

```
File not found. Check the filename and the current folder.
Requested path: missing_S09_orders.csv
```

Catch only `FileNotFoundError` (Session 08: keep the `except` specific). Then check: is the name spelled right, is the extension right, and is the file in the folder VS Code has open?

TRY IT YOURSELF

Write `cash_up.txt` containing the line "Till 1 balanced". Then open it again with mode `"a"` and append "\nTill 2 balanced". Finally read the file and print it. What would have happened if you had used `"w"` the second time?

Part 7 CSV: rows become dictionaries

A **CSV** (comma-separated values) file is how spreadsheets and databases usually export data. `S09_orders.csv` holds ten orders with eight fields:

order_id	customer_name	region	product_name	unit_price	qty	disc %	tax %
ORD-1001	Northstar Retail	West	Analytics Starter Kit	250.0	4	10.0	5.0
ORD-1002	Bluewave Studio	North	Analytics Starter Kit	250.0	5	10.0	5.0
ORD-1003	Greenfield Foods	South	Data Visualization Pack	180.0	3	0.0	5.0
ORD-1004	Urban Market Lab	West	Forecasting Toolkit	420.0	3	5.0	12.0
ORD-1005	Summit Services	East	Analytics Starter Kit	250.0	2	0.0	5.0
ORD-1006	Coastal Commerce	North	Data Visualization Pack	180.0	6	8.0	5.0
ORD-1007	Metro Office Co	South	Customer Insights Bundle	320.0	3	12.5	5.0
ORD-1008	Horizon Learning	East	Forecasting Toolkit	420.0	2	0.0	12.0
ORD-1009	Cedar Supply Co	West	Customer Insights Bundle	320.0	4	5.0	8.0
ORD-1010	Brightpath Consulting	North	Workflow Essentials	95.0	8	10.0	5.0

PREDICT FIRST

Which of the eight fields look numeric? After Python loads the file, what type do you think `unit_price` will be?

Loading with csv.DictReader

p7_load.py

```
import csv

raw_orders = []

with open("S09_orders.csv", mode="r", encoding="utf-8", newline="") as csv_file:
    reader = csv.DictReader(csv_file)
    for row in reader:
        raw_orders.append(row)

print("Record count:", len(raw_orders))
print("First raw record:", raw_orders[0])
print("Raw unit_price type:", type(raw_orders[0]["unit_price"]))
```

OUTPUT

```
Record count: 10
First raw record: {'order_id': 'ORD-1001', 'customer_name': 'Northstar Retail', 'region':
'West', 'product_name': 'Analytics Starter Kit', 'unit_price': '250.0', 'quantity': '4',
'discount_pct': '10.0', 'tax_pct': '5.0'}
Raw unit_price type: <class 'str'>
```

`import csv` makes a standard-library tool available (modules come in Session 10). `DictReader` uses the header row as the **keys**, so every row becomes a dictionary and `raw_orders` is a list of dictionaries. `newline=""` lets the csv module handle line endings.

> **Watch out: A numeric-looking value in a CSV is still text.** `"250.0"` must be converted before it behaves like the number 250.0, exactly as with `input()` in Session 07.

Converting the numeric fields

p7_convert.py

```
def convert_numeric_fields(raw_order):
    order = raw_order.copy()
    order["unit_price"] = float(order["unit_price"])
    order["quantity"] = int(order["quantity"])
    order["discount_pct"] = float(order["discount_pct"])
    order["tax_pct"] = float(order["tax_pct"])
    return order

first = convert_numeric_fields(raw_orders[0])
print(first)
print(type(first["unit_price"]), type(first["quantity"]))
print("raw still text:", type(raw_orders[0]["unit_price"]))
```

OUTPUT

```
{'order_id': 'ORD-1001', 'customer_name': 'Northstar Retail', 'region': 'West',
'product_name': 'Analytics Starter Kit', 'unit_price': 250.0, 'quantity': 4,
'discount_pct': 10.0, 'tax_pct': 5.0}
<class 'float'> <class 'int'>
raw still text: <class 'str'>
```

`.copy()` makes a new dictionary, so `raw_orders` stays exactly as read and you can compare the two. Prices and percentages become `float`; quantity is a whole number, `int`.

When a value cannot be converted

p7_valueerror.py

```
edited_quantity = "not-a-number"

try:
    int(edited_quantity)
except ValueError:
    print("Conversion failed: quantity must contain whole-number text.")
```

OUTPUT

```
Conversion failed: quantity must contain whole-number text.
```

Enrich each record with the Session 08 calculation

Each converted order gets two derived fields: `final_total` (same formula as Session 08) and `order_band` (large at 950.0 or more). The two anchors from Session 08 reappear:

	ORD-1001 (quantity 4)	ORD-1002 (quantity 5)
subtotal	250.0 × 4 = 1000.0	250.0 × 5 = 1250.0
after 10% discount	900.0	1125.0
after 5% tax = final_total	945.0	1181.25
order_band	standard	large

Writing a CSV (awareness)

p7_writer.py

```
import csv

fieldnames = ["order_id", "region", "order_band"]
rows_to_write = [
    {"order_id": "ORD-1001", "region": "West", "order_band": "standard"},
    {"order_id": "ORD-1002", "region": "North", "order_band": "large"},
]

with open("order_bands.csv", mode="w", encoding="utf-8", newline="") as csv_file:
    writer = csv.DictWriter(csv_file, fieldnames=fieldnames)
    writer.writeheader()
    writer.writerows(rows_to_write)

with open("order_bands.csv", mode="r", encoding="utf-8") as f:
    print(f.read())
```

OUTPUT

```
order_id,region,order_band
ORD-1001,West,standard
ORD-1002,North,large
```

TRY IT YOURSELF

Load `S09_orders.csv` with `DictReader`. Loop through the rows and add up the total **quantity** across all ten orders. (Hint: convert each quantity with `int()` before adding.) Then print how many orders came from the North region.

Part 8 JSON: structured data other programs can read

CSV suits flat rows and columns. **JSON** (JavaScript Object Notation) also keeps nested structure, such as lists of records, so it is the usual format for APIs and for passing data between programs.

How JSON maps to Python

JSON	Python	In the order data
object `{...}`	`dict`	one order record
array `[...]`	`list`	the collection of orders
string	`str`	customer_name, order_band
number	`int` or `float`	quantity, unit_price, final_total
`true` / `false`	`True` / `False`	not in this dataset
`null`	`None`	not in this dataset
(no set type)	`set`	convert to a list before saving

Two directions: parse and serialise

Function	Direction	Works with
`json.loads(text)`	JSON text → Python	a string
`json.load(file)`	JSON text → Python	an open file
`json.dumps(obj)`	Python → JSON text	a string
`json.dump(obj, file)`	Python → JSON text	an open file

Read the trailing **s** as *string*.

p8_loads.py

```
import json

json_text = '{"order_id": "DEMO-01", "quantity": 4, "active": true, "notes": null}'
record = json.loads(json_text)

print(record)
print(record["quantity"] + 1)
```

OUTPUT

```
{'order_id': 'DEMO-01', 'quantity': 4, 'active': True, 'notes': None}
5
```

The JSON number became a real Python integer, `true` became `True` and `null` became `None`.

Saving the enriched orders

p8_dump.py

```
import json

with open("S09_enriched_orders.json", mode="w", encoding="utf-8") as json_file:
    json.dump(orders, json_file, indent=2)

with open("S09_enriched_orders.json", mode="r", encoding="utf-8") as json_file:
    lines = json_file.read().splitlines()

print("\n".join(lines[:13]))
print("  ...")
```

OUTPUT

```
[
  {
    "order_id": "ORD-1001",
    "customer_name": "Northstar Retail",
    "region": "West",
    "product_name": "Analytics Starter Kit",
    "unit_price": 250.0,
    "quantity": 4,
    "discount_pct": 10.0,
    "tax_pct": 5.0,
    "final_total": 945.0,
    "order_band": "standard"
  },
  ...
```

`indent=2` makes the file readable for people. Numbers are saved without quotes, so they come back as numbers. The print shows only the first record of the file.

A controlled failure: sets are not JSON

p8_set_error.py

```
import json

unique_regions = {"West", "North", "South", "East"}
print(json.dumps(unique_regions))
```

ERROR

```
Traceback (most recent call last):
  File "C:\Users\student\s09_data_structures\p8_set_error.py", line 4, in <module>
    print(json.dumps(unique_regions))
          ~~~~~~~~~~^^^^^^^^^^^^^^^^
TypeError: Object of type set is not JSON serializable
```

p8_set_fixed.py

```
import json

unique_regions = {"West", "North", "South", "East"}
print(json.dumps(sorted(unique_regions)))
```

OUTPUT

```
["East", "North", "South", "West"]
```

Rule: convert to a JSON-friendly type before saving. `sorted()` turns the set into a list, in the same order every time.

A summary for people

p8_summary.py

```
summary_lines = [
    f"Total orders processed: {len(orders)}",
    f"Large orders: {len(large_orders)}",
    f"Unique regions: {', '.join(sorted(unique_regions))}",
]

with open("S09_summary.txt", mode="w", encoding="utf-8") as summary_file:
    summary_file.write("\n".join(summary_lines))

with open("S09_summary.txt", mode="r", encoding="utf-8") as summary_file:
    print(summary_file.read())
```

OUTPUT

```
Total orders processed: 10
Large orders: 4
Unique regions: East, North, South, West
```

Same data, different reader: the JSON file is for programs (complete records that can be loaded back), the text file is for people. `"\n".join(...)` puts each line on its own row; `", ".join(...)` joins the regions with commas.

The round trip: reload and verify

p8_reload.py

```
import json

with open("S09_enriched_orders.json", mode="r", encoding="utf-8") as json_file:
    reloaded_orders = json.load(json_file)

print("Records:", len(reloaded_orders))
print(type(reloaded_orders), type(reloaded_orders[0]), type(reloaded_orders[0]
["quantity"]))
print(reloaded_orders[1]["order_id"], reloaded_orders[1]["final_total"], reloaded_orders[1]
["order_band"])
```

OUTPUT

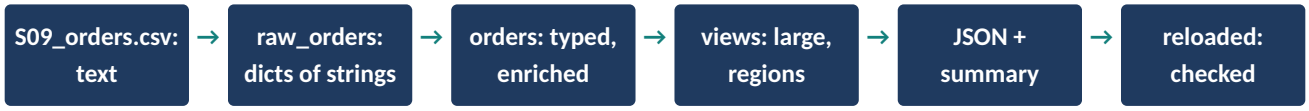
```
Records: 10
<class 'list'> <class 'dict'> <class 'int'>
ORD-1002 1181.25 large
```

A file can exist and still be wrong: missing records, a wrong band, numbers saved as text. **Reload it and test it.**

> TRY IT YOURSELF
>
> Create a dictionary `report` with two keys: `"large_order_ids"` (the list from Part 5) and `"regions"` (the sorted list of unique regions). Save it to `report.json` with `indent=2`, reload it into a new variable, and print `reloaded["regions"]`. What would go wrong if you saved the set without `sorted()`?

Part 9 Integrated practical: Sales Order Records mini-pipeline

> **Trainer note:** Learners complete `s09_pipeline_starter.py`. Coach rather than lecture. If time is short, protect the round trip: JSON written, reloaded and checked.



Checkpoint	You do	You check
1 · Inspect	Read the eight headers; decide which only look numeric	your prediction, before running
2 · Load	Complete the DictReader loop into `raw_orders`; do not convert yet	count 10; `unit_price` is `str`
3 · Convert and enrich	In `convert_and_enrich()`: copy, convert four fields, add `final_total` and `order_band`	count 10; `float`, `int`, `float`
4 · Derive views	`large_orders` (loop), `unique_regions` (set comprehension), `large_order_ids` (list comprehension)	4 large, 6 standard
5 · Save JSON	`json.dump` to `S09_enriched_orders.json`	the file opens as indented text
6 · Summary	Three lines to `S09_summary.txt`	total, large count, sorted regions
7 · Reload	`json.load` into `reloaded_orders`	count, classes, anchors, types
8 · Clean run	Delete the two output files, run the script from the top	everything is recreated and passes

Expected enriched orders

order_id	customer_name	region	final_total	order_band
ORD-1001	Northstar Retail	West	945.00	standard
ORD-1002	Bluewave Studio	North	1181.25	large
ORD-1003	Greenfield Foods	South	567.00	standard
ORD-1004	Urban Market Lab	West	1340.64	large
ORD-1005	Summit Services	East	525.00	standard
ORD-1006	Coastal Commerce	North	1043.28	large
ORD-1007	Metro Office Co	South	882.00	standard
ORD-1008	Horizon Learning	East	940.80	standard
ORD-1009	Cedar Supply Co	West	1313.28	large
ORD-1010	Brightpath Consulting	North	718.20	standard

Values are displayed to two decimals; the stored numbers are not rounded (ORD-1004 is stored as `1340.6399999999999`, a normal floating-point effect).

Reference run

A completed pipeline prints this. If yours differs, compare in this order: record count, class counts, anchors, then types.

TERMINAL

```
> python s09_pipeline.py
Record count: 10
Raw unit_price type: <class 'str'>
Enriched record count: 10
Converted types: <class 'float'> <class 'int'>
Large-order count: 4
Standard-order count: 6
Large-order IDs: ['ORD-1002', 'ORD-1004', 'ORD-1006', 'ORD-1009']
Unique regions (stable display): ['East', 'North', 'South', 'West']
Record count verified: 10
Large / standard counts: 4 / 6
Standard anchor verified: 945.0 standard
Large anchor verified: 1181.25 large
Numeric fields reloaded as numbers: True
```

S09_SUMMARY.TXT

```
Total orders processed: 10
Large orders: 4
Unique regions: East, North, South, West
```

Tip: The verification uses `all(...)`, which is True only if the check is True for every record. It is a compact way to say "every reloaded order has numeric fields". `isinstance(value, float)` asks whether a value is of that type.

Close: one lifecycle

Text came in, structure lived inside the program, text went out, and the result was verified.

Where each structure earned its place	Where each boundary needed care
Dictionary: one named order	**CSV:** everything arrives as text, so convert
List: the ordered collection of orders	**JSON:** no set type, so save a sorted list
Set: the unique regions	**Reload:** trust the saved file only after checks
Comprehension: a short filter or transform	

Ten questions to ask of any data problem

1. What is one record here?
2. Which fields does every record have?
3. Does order matter, and can values repeat?
4. Which structure fits that need?
5. Where does the data come from?
6. What type is each value when it arrives?
7. What must be converted or derived?
8. Who or what will read the output?
9. Which format fits that reader?
10. How will I check the saved result?

Reflection

- Why is a list of dictionaries a good shape for order records?
- Why do CSV values need conversion after loading?
- When did a set help, and why could it not go straight into JSON?
- When is a loop clearer than a comprehension?
- What does the JSON file keep that the summary file does not?
- Which check proves the round trip worked?

Next: Session 10 covers modules, project structure, classes and Git; Sessions 11–12 bring NumPy and Pandas, which build directly on lists and lists of dictionaries.

Appendix: Answers to the exercises

Part 1

a1.py

```
monthly_sales = [42000, 38500, 51000, 47250]

print(monthly_sales[0], monthly_sales[-1])
print(monthly_sales[1:3])

monthly_sales[1] = 39000
monthly_sales.append(49800)

for sales in monthly_sales:
    print(sales)
print("Total:", sum(monthly_sales))
```

OUTPUT

```
42000 47250
[38500, 51000]
42000
39000
51000
47250
49800
Total: 229050
```

Part 2

a2.py

```
employee = {"employee_id": "E-118", "name": "Sipho Dlamini",
            "department": "Finance", "salary": 32000.0}

print(employee["department"])
employee["salary"] = employee["salary"] * 1.05
employee["manager"] = "L. Mokoena"
print(employee.get("phone", "Not on file"))
print(employee)
```

OUTPUT

```
Finance
Not on file
{'employee_id': 'E-118', 'name': 'Sipho Dlamini', 'department': 'Finance', 'salary':
33600.0, 'manager': 'L. Mokoena'}
```

Part 3

a3.py

```
accounts = [
    {"name": "Thandi", "balance": 15250.0},
    {"name": "Pieter", "balance": 8400.0},
    {"name": "Ayesha", "balance": 22100.0},
]

for account in accounts:
    print(account["name"], account["balance"])

for account in accounts:
    if account["balance"] > 10000:
        print("Above 10 000:", account["name"])

total = 0.0
for account in accounts:
    total = total + account["balance"]
print("Combined balance:", total)
```

OUTPUT

```
Thandi 15250.0
Pieter 8400.0
Ayesha 22100.0
Above 10 000: Thandi
Above 10 000: Ayesha
Combined balance: 45750.0
```

Part 4

Need	Structure	Reason
(1) every card swipe, in order	list	keeps order and allows repeats (same amount twice)
(2) one employee's details	dictionary	each field is read by a meaningful name
(3) a branch's (latitude, longitude)	tuple	a small fixed pair that should not change
(4) distinct ATM IDs with errors	set	keeps each ATM once; easy membership checks

Part 5

a5.py

```
accounts = [
    {"name": "Thandi", "balance": 15250.0, "segment": "Premier"},
    {"name": "Pieter", "balance": 8400.0, "segment": "Standard"},
    {"name": "Ayesha", "balance": 22100.0, "segment": "Premier"},
]

print([account["name"].upper() for account in accounts])
print([account["name"] for account in accounts if account["balance"] > 10000])
print(sorted({account["segment"] for account in accounts}))
```

OUTPUT

```
['THANDI', 'PIETER', 'AYESHA']
['Thandi', 'Ayesha']
['Premier', 'Standard']
```

Part 6

a6.py

```
with open("cash_up.txt", mode="w", encoding="utf-8") as f:
    f.write("Till 1 balanced")

with open("cash_up.txt", mode="a", encoding="utf-8") as f:
    f.write("\nTill 2 balanced")

with open("cash_up.txt", mode="r", encoding="utf-8") as f:
    print(f.read())
```

OUTPUT

```
Till 1 balanced
Till 2 balanced
```

With `"w"` the second time, the first line would have been replaced and only "Till 2 balanced" would remain.

Part 7

a7.py

```
import csv

total_quantity = 0
north_orders = 0

with open("S09_orders.csv", mode="r", encoding="utf-8", newline="") as csv_file:
    for row in csv.DictReader(csv_file):
        total_quantity = total_quantity + int(row["quantity"])
        if row["region"] == "North":
            north_orders = north_orders + 1

print("Total quantity:", total_quantity)
print("North orders:", north_orders)
```

OUTPUT

```
Total quantity: 40
North orders: 3
```

Part 8

a8.py

```python
import json

report = {
    "large_order_ids": ["ORD-1002", "ORD-1004", "ORD-1006", "ORD-1009"],
    "regions": sorted({"West", "North", "South", "East"}),
}

with open("report.json", mode="w", encoding="utf-8") as f:
    json.dump(report, f, indent=2)

with open("report.json", mode="r", encoding="utf-8") as f:
    reloaded = json.load(f)

print(reloaded["regions"])
```

OUTPUT

```
['East', 'North', 'South', 'West']
```

Saving the set directly raises `TypeError: Object of type set is not JSON serializable`.

Part 9: completed s09_pipeline.py

s09_pipeline_solution.py

```python
# Session 09 -- Sales Order Records mini-pipeline (completed)
import csv
import json

INPUT_FILE = "S09_orders.csv"
ENRICHED_JSON_FILE = "S09_enriched_orders.json"
SUMMARY_FILE = "S09_summary.txt"
LARGE_ORDER_THRESHOLD = 950.0


# ---- Supplied helpers (from Session 08) ----
def calculate_final_total(unit_price, quantity, discount_pct, tax_pct):
    subtotal = unit_price * quantity
    discounted_subtotal = subtotal - subtotal * discount_pct / 100
    return discounted_subtotal + discounted_subtotal * tax_pct / 100


def classify_order(final_total, threshold=LARGE_ORDER_THRESHOLD):
    if final_total >= threshold:
        return "large"
    return "standard"


# ---- Checkpoint 2: load ----
def load_orders(path):
    raw_orders = []
    with open(path, mode="r", encoding="utf-8", newline="") as csv_file:
        reader = csv.DictReader(csv_file)
        for row in reader:
            raw_orders.append(row)
    return raw_orders
```

```
# ---- Checkpoint 3: convert and enrich ----
def convert_and_enrich(raw_order):
    order = raw_order.copy()
    order["unit_price"] = float(order["unit_price"])
    order["quantity"] = int(order["quantity"])
    order["discount_pct"] = float(order["discount_pct"])
    order["tax_pct"] = float(order["tax_pct"])
    order["final_total"] = calculate_final_total(
        order["unit_price"], order["quantity"], order["discount_pct"], order["tax_pct"]
    )
    order["order_band"] = classify_order(order["final_total"])
    return order


def main():
    raw_orders = load_orders(INPUT_FILE)
    print("Record count:", len(raw_orders))
    print("Raw unit_price type:", type(raw_orders[0]["unit_price"]))

    orders = []
    for raw_order in raw_orders:
        orders.append(convert_and_enrich(raw_order))
    print("Enriched record count:", len(orders))
    print("Converted types:", type(orders[0]["unit_price"]), type(orders[0]["quantity"]))

    # ---- Checkpoint 4: derived views ----
    large_orders = []
    for order in orders:
        if order["final_total"] >= LARGE_ORDER_THRESHOLD:
            large_orders.append(order)
    unique_regions = {order["region"] for order in orders}
    large_order_ids = [order["order_id"] for order in large_orders]

    print("Large-order count:", len(large_orders))
    print("Standard-order count:", len(orders) - len(large_orders))
    print("Large-order IDs:", large_order_ids)
    print("Unique regions (stable display):", sorted(unique_regions))

    # ---- Checkpoint 5: save JSON ----
    with open(ENRICHED_JSON_FILE, mode="w", encoding="utf-8") as json_file:
        json.dump(orders, json_file, indent=2)

    # ---- Checkpoint 6: summary for people ----
    summary_lines = [
        f"Total orders processed: {len(orders)}",
        f"Large orders: {len(large_orders)}",
        f"Unique regions: {', '.join(sorted(unique_regions))}",
    ]
    with open(SUMMARY_FILE, mode="w", encoding="utf-8") as summary_file:
        summary_file.write("\n".join(summary_lines))

    # ---- Checkpoint 7: reload and verify ----
    with open(ENRICHED_JSON_FILE, mode="r", encoding="utf-8") as json_file:
        reloaded_orders = json.load(json_file)

    by_id = {order["order_id"]: order for order in reloaded_orders}
    large_count = len([o for o in reloaded_orders if o["order_band"] == "large"])
    numbers_ok = all(
        isinstance(o["unit_price"], float) and isinstance(o["quantity"], int)
        for o in reloaded_orders
```

```
    )

    assert len(reloaded_orders) == 10
    assert large_count == 4
    assert by_id["ORD-1001"]["final_total"] == 945.0
    assert by_id["ORD-1001"]["order_band"] == "standard"
    assert by_id["ORD-1002"]["final_total"] == 1181.25
    assert by_id["ORD-1002"]["order_band"] == "large"
    assert numbers_ok

    print("Record count verified:", len(reloaded_orders))
    print("Large / standard counts:", large_count, "/", len(reloaded_orders) - large_count)
    print("Standard anchor verified:", by_id["ORD-1001"]["final_total"], by_id["ORD-1001"]
["order_band"])
    print("Large anchor verified:", by_id["ORD-1002"]["final_total"], by_id["ORD-1002"]
["order_band"])
    print("Numeric fields reloaded as numbers:", numbers_ok)


if __name__ == "__main__":
    main()
```